

WorkHub

Enterprise Team Collaboration Platform

Project Planning & Technical Documentation

Purpose: Define what WorkHub is, why it is being built, its requirements, features, technology choices, architecture direction, development roadmap, folder structure, and professional Git/GitHub workflow.

1. Project Overview

What is WorkHub? WorkHub is a production-quality enterprise team collaboration platform that combines project management, task tracking, team communication, and productivity analytics in one workspace.

Inspiration: Jira, Notion, Slack, GitHub Projects, and Linear.

2. Problem Statement

Teams often use separate tools for planning, task management, communication, file sharing, and progress tracking. This creates fragmented workflows. WorkHub aims to provide a unified workspace where teams can manage work and communicate in one place.

3. Project Goals

- Master the MERN stack through a realistic production-style application.
- Understand backend architecture instead of copying code.
- Practice authentication, authorization, validation, security, APIs, realtime communication, and deployment.
- Build a professional GitHub repository suitable for a portfolio.
- Learn professional Git, GitHub Issues, Projects, Pull Requests, Releases, and CI.
- Develop clean, maintainable, scalable code and strong debugging skills.

4. Target Users & Roles

Role	Main Responsibilities
Guest	Landing page, registration, login
Member	Work on tasks, comment, update profile
Project Manager	Manage projects, assign tasks, view reports
Organization Admin	Manage members, roles, projects, organization settings

5. Core Features

Authentication: Registration, Login / Logout, JWT access tokens, Refresh tokens, HTTP-only cookies, Email verification, Password reset

Users: Profile, Avatar, Skills, Bio / preferences, Settings

Organizations: Create organization, Teams, Invitations, Roles, Permissions

Projects: CRUD, Validation, Pagination, Filtering, Archive

Kanban: Backlog, Todo, In Progress, Review, Done, Drag & drop

Tasks: CRUD, Assignment, Comments, Attachments, Labels, Priority, Due dates

Realtime: Socket.io, Team chat, Direct messages, Online status, Typing indicators, Notifications

Dashboard: Project overview, Task statistics, Productivity metrics, Charts, Team activity

Search: Global search, Debouncing, MongoDB indexing

6. Functional Requirements

- Users can create accounts and authenticate securely.
- Authenticated users can manage their profiles.
- Organizations support teams, members, roles, and permissions.
- Authorized users can create and manage projects.
- Projects support Kanban-based task management.
- Tasks support assignment, comments, labels, priority, due dates, and attachments.
- Users receive relevant notifications and can communicate through realtime chat.
- Users can search relevant WorkHub data.
- Authorized users can view project and productivity analytics.

7. Non-Functional Requirements

- Security
- Performance
- Scalability
- Maintainability
- Reliability
- Responsive UI
- Error handling and logging
- Separation of concerns
- Input validation
- Production-ready configuration

8. Technology Stack

Technology	Purpose / Why
React	Component-based frontend and UI.
React Router	Client-side application routing.
Redux Toolkit	Complex shared client state.
Tailwind CSS	Consistent responsive UI development.
Axios	HTTP communication with backend APIs.
React Hook Form	Efficient form state and validation integration.
Node.js	JavaScript runtime for backend.
Express.js	HTTP server and REST API framework.
MongoDB	Flexible document database for collaboration data.
Mongoose	MongoDB schema modeling and data access.
JWT	Access-token authentication.
HTTP-only Cookies	Safer browser handling for refresh/session-related tokens.
Socket.io	Realtime chat, presence, and notifications.
Zod	Schema-based validation.
Multer	Multipart file uploads.
Cloudinary	Media storage and delivery.
Git + GitHub	Version control, collaboration, review, issues, releases, CI.
Postman / Bruno	API development and testing.
ESLint + Prettier	Code quality and consistent formatting.

9. High-Level Architecture

Browser → React Client → REST API → Express → Business Logic → Mongoose → MongoDB

Realtime: React Client ↔ Socket.io ↔ Node/Express server

The application will use separation of concerns: routes handle routing, controllers coordinate requests, services contain business logic, models represent database data, middleware handles cross-cutting concerns, and configuration manages environment-specific settings.

10. Planned Backend Architecture

```
server/  
■■■■ src/  
■ ■■■■ config/  
■ ■■■■ controllers/  
■ ■■■■ middlewares/  
■ ■■■■ models/  
■ ■■■■ routes/  
■ ■■■■ services/  
■ ■■■■ validators/  
■ ■■■■ utils/  
■ ■■■■ app.js  
■ ■■■■ server.js  
■■■■ tests/
```

This is a starting architecture. It will evolve when real requirements justify better boundaries.

11. Repository Structure

```
workhub/  
■■■■ client/  
■■■■ server/  
■■■■ docs/  
■■■■ .gitignore  
■■■■ README.md  
■■■■ LICENSE
```

12. Development Phases

Phase	Area	Scope
1	Planning	Requirements, features, user stories, tech stack, architecture, Git setup
2	Backend Setup	Express, MongoDB, MVC, environment variables, configuration, error handling
3	Authentication	JWT, refresh tokens, cookies, password reset, email verification
4	Users	Profile, avatar, skills, settings
5	Organizations	Teams, roles, permissions
6	Projects	CRUD, validation, pagination, filtering
7	Kanban	Drag & drop, task movement
8	Tasks	Comments, attachments, labels, priority
9	Realtime	Socket.io, chat, notifications
10	Dashboard	Analytics, charts
11	Search	Debouncing, indexing
12	Deployment	Docker, production build, environment variables

13. Git & GitHub Strategy

- main: stable, production-ready code.
- Feature branches: one focused change per branch.
- Pull Requests: describe what changed, why, and how it was tested.
- Issues: track features, bugs, technical debt, and improvements.
- GitHub Projects: manage backlog and milestones.
- Releases and tags: mark meaningful stable versions.
- GitHub Actions: run CI checks such as linting, tests, and builds.
- Avoid meaningless commits such as 'update', 'changes', or 'final'.

14. Conventional Commits

Format: type(scope): description

- feat(auth): implement JWT login endpoint
- fix(tasks): resolve task update validation bug
- docs(readme): add installation instructions

- refactor(users): extract profile service
- chore(project): initialize repository structure

The commit type communicates intent, the scope identifies the affected area, and the description clearly states the change.

15. README Plan

- Project Overview
- Features
- Architecture Diagram
- Folder Structure
- Tech Stack
- Installation
- Environment Variables
- Screenshots
- API Documentation
- Future Improvements
- License

16. Engineering Quality Principles

- Clean Code
- SOLID where applicable
- DRY without over-abstraction
- KISS
- Separation of Concerns
- Reusable Components
- Input Validation
- Authentication & Authorization
- Secure secret handling
- Performance optimization
- Logging and centralized error handling
- Tests for important business logic

17. Development Rules

- Build incrementally; do not generate giant project files.
- Understand important concepts before implementation.
- Diagnose errors before applying fixes.
- Review and refactor working beginner implementations toward better production approaches.
- End coding sessions with meaningful Conventional Commits.

- After major modules, use theory, coding, practical, and architecture interview questions to evaluate understanding.

18. Version 1 Scope

Version 1 prioritizes the core collaboration workflow: authentication → users → organizations → projects → Kanban → tasks → realtime collaboration → dashboard → search → deployment.

Video conferencing, billing/subscriptions, advanced AI assistants, and complex enterprise integrations are intentionally outside the initial scope.

19. Immediate Next Steps

- Create the GitHub repository: workhub.
- Clone it locally.
- Create client, server, and docs directories.
- Create .gitignore, README.md, and LICENSE.
- Create docs/project-overview.md and docs/requirements.md.
- Review the requirements before application coding.
- First commit: chore(project): initialize WorkHub repository structure

Mentorship Principle

The goal is not merely to make WorkHub work. The goal is to understand why every architectural and coding decision is made.